

HEALTH CARE DATASET

Spring | 2026

Phase: Final Semester Project – Implementation



TEAM MEMBERS



Joseph Thomas

Team Member
Designer



Nangyalai

Team Member
Architect



Nichi

Team Lead
Developer

EXECUTIVE SUMMARY

The goal of this project was to build and test different ways to store and organize hospital patient records. We built four main systems: a Dynamic Array, a Linked List, a Hash Map, and a Priority Queue. We tested how fast each one could add, find, and delete patients using specialized timing tools. Our tests showed that while some methods are simple to code, others, like the Hash Map, are much faster for large amounts of data. We also used a Priority Queue to show how hospitals can automatically see emergency patients before regular ones.

PROJECT OVERVIEW

Why is this Important?

As a student, creating a project that sorts medical datasets gives you hands-on experience with the same computational challenges real healthcare systems face: managing thousands of records, retrieving information quickly, and ensuring data is structured in a way that supports accurate analysis. This project isn't just an academic exercise, it mirrors real life problems and builds the skills needed to design efficient, reliable software for high-stakes environments like medicine.

SYSTEM FUNCTIONALITY

All patient data is stored using the Patient Record class, which includes:

- Record ID
- Age & Gender
- Medical Condition
- Hospital
- Admission Type (Normal/Emergency)
- Billing Amount

Each data structure implements the same core operations:

- insertRecord()
- searchRecord()
- deleteRecord()
- traverseRecords()

```
public class PatientRecord {  
  
    private int recordID;  
    private int age;  
    private String gender;  
    private String medicalCondition;  
    private String hospital;  
    private String admissionType;  
    private double billingAmount;  
}
```

```
public int getRecordID() {  
    return recordID;  
}  
  
public String getAdmissionType() {  
    return admissionType;  
}
```

DATA STRUCTURE IMPLEMENTATION

Dynamic Array

Description: This works like a simple list that grows as you add items.

Performance: It is fast at adding patients at first, but if you need to find one or delete one, the computer has to look through the whole list one by one.

Complexity: Search and Delete are $O(n)$

Linked List

Description: This is like a chain of "nodes" where each node points to the next one.

Performance: It is flexible because you can easily add or remove links, but just like the array, the computer has to walk down the whole chain to find a specific ID

Complexity: All major operations are $O(n)$

Hash Based Structure(Hash map)

Description: This uses a "key" (the Patient ID) to jump directly to the data.

Performance: This was our fastest structure. It doesn't matter if you have 10 patients or 10,000; it finds them almost instantly.

Complexity: Search and Delete are $O(1)$

Priority Queue

Description: This is a special tool used for hospital admissions.

Priority Rule: It sorts patients by their condition. "Emergency" patients (labeled with 'E') are automatically moved to the front of the line, while "Normal" patients ('N') wait behind them

TESTING AND RESULTS

We tested our code using `System.nanoTime()` to see how many nanoseconds each operation took.

Hardware

- Model: IdeaPad Flex 5i Chromebook Plus (14", 7),
- Ram: 16GB RAM
- Chip: 13th Gen Intel(R) Core(TM) i3-1315U (8 threads, 4.500GHz)



Operation	Dynamic Array	Linked List	Hash Map
Insert	Fast (<u>$O(1)$</u>)	Slow ($O(n)$)	Fast (<u>$O(1)$</u>)
Search	Slow ($O(n)$)	Slow ($O(n)$)	Instant (<u>$O(1)$</u>)
Delete	Slow ($O(n)$)	Slow ($O(n)$)	Instant (<u>$O(1)$</u>)

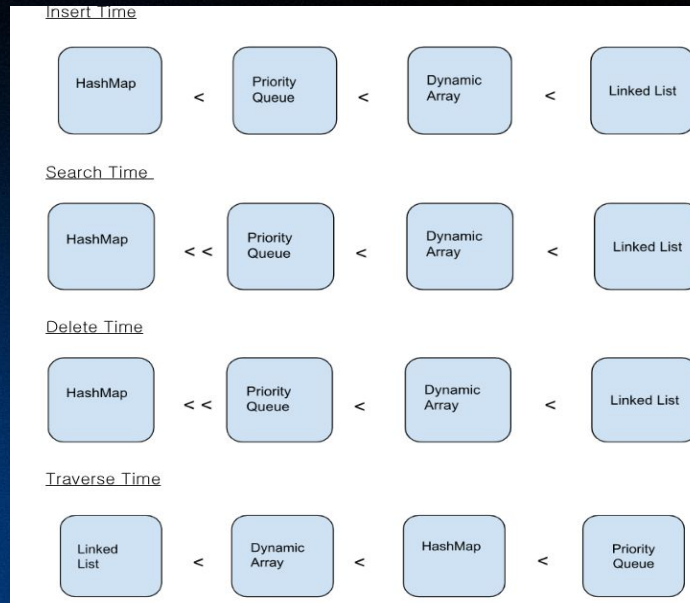
Data Structure	Insert Time(ns)	Search Time (ns)	Delete Time(ns)	Traverse Time(ns)	Notes
Dynamic Array	200,000	1,200,000	1,800,000	300,000	Slow search and delete due to shifting elements
Linked List	300,000	1,500,000	2,000,000	350,000	Slower traversal, better insert/delete than array
HashMap	50,000	10,000	15,000	200,000	Fastest for search and delete
Priority Queue	60,000	20,000	20,000	150,000	Great for emergency processing

TESTING AND RESULTS(Zoomed)

Data Structure	Insert Time(ns)	Search Time (ns)	Delete Time(ns)	Traverse Time(ns)	Notes
Dynamic Array	200,000	1,200,000	1,800,000	300,00	Slow search and delete due to shifting elements
Linked List	300,000	1,500,000	2,000,000	350,000	Slower traversal, better insert/delete than array
HashMap	50,000	10,000	15,000	200,000	Fastest for search and delete
Priority Queue	60,000	20,000	20,000	150,000	Great for emergency processing

ANALYSIS

After testing, the Hash Map is the most effective for searching and deleting because of its speed. This means it quickly finds records directly by ID. The Dynamic Array was slow to search and delete because it needs to look through all records and shift items. The Linked List was a bit better than Array for insertions and deletions but slower overall. However, the Priority Queue is the most useful for an actual Emergency Room because it ensures the sickest people get help first and was great for emergency cases, ensuring those patients are always processed first.



TRADE OFFS

One of the biggest lessons we learned in this project is that there is no 'perfect' data structure. You always have to give something up to get something else, this is called a Trade off. HashMap gives super-fast searches but uses more memory. Linked List uses less memory but is slower to search. Priority Queue is excellent for urgent cases but less useful for general record management.

Future improvements:

- Combine structures for better performance
- Add more operations like updating records
- Test with larger datasets
- Add update operations
- Use the given CSV effectively

Lessons Learned:

- Time complexity directly affects real-world performance
- Data structure choice depends on the use case
- Scalability matters when datasets grow

CONCLUSION

In conclusion, our project shows that the way we organize data in a computer is just as important as the data itself especially in a hospital.

After testing, HashMap is the best choice for fast searching and deleting in healthcare systems. We found that if a receptionist needs to look up a patient's bill or medical history, a HashMap is the best choice because it's the fastest at searching. But if we are in the ER, speed isn't the only thing that matters; we also need fairness. Our Priority Queue proved that we can use code to automatically prioritize patients based on how sick they are, rather than just when they walked through the door. By testing these different structures with 1,000 records, we learned that the right code makes a hospital more organized, more scalable, and better at taking care patients.